PIMPRI CHINCHWAD EDUCATION TRUST's

PIMPRI CHINCHWAD COLLEGE OF ENGINEERING

(An Autonomous Institute)

DATABASE MANAGEMENT SYSTEM LABORATORY

NAME	Viraj Pathare
PRN	124B1F149
CLASS	SY - IT
DIV	A

1. Problem Statement

Write cursor code using control structures and exception handling.

2. Creation of Database:

```
mysql> CREATE DATABASE StreamFlixDB;
ERROR 1007 (HY000): Can't create database 'streamflixdb'; database exists
```

```
mysql> USE StreamFlixDB;
Database changed
```

2. Table Creation and Sample Data

```
CREATE DATABASE IF NOT EXISTS StreamingDB;
USE StreamingDB;

CREATE TABLE User (
user_id INT AUTO_INCREMENT PRIMARY KEY,
name VARCHAR(100) NOT NULL,
email VARCHAR(150) NOT NULL UNIQUE,
phone VARCHAR(15)
);

CREATE TABLE Subscription_Plan (
plan_id INT AUTO_INCREMENT PRIMARY KEY,
plan_name VARCHAR(50) NOT NULL,
price DECIMAL(8,2) NOT NULL
);

CREATE TABLE Subscription (
subscription_id INT AUTO_INCREMENT PRIMARY KEY,
start_date DATE,
end_date DATE,
status VARCHAR(20) DEFAULT 'Active',
user_id INT,
plan_id INT,
FOREIGN KEY (user_id) REFERENCES User(user_id),
FOREIGN KEY (plan_id) REFERENCES Subscription_Plan(plan_id)
);

CREATE TABLE Payment (
payment_id INT AUTO_INCREMENT PRIMARY KEY,
payment_date DATE,
amount DECIMAL(8,2),
```

```
payment_method VARCHAR(20),
payment_status VARCHAR(20),
subscription_id INT,
FOREIGN KEY (subscription_id) REFERENCES Subscription(subscription_id) );

CREATE TABLE Content (
content_id INT AUTO_INCREMENT PRIMARY KEY,
title VARCHAR(200) NOT NULL,
genre VARCHAR(50),
duration INT
);

CREATE TABLE Watch_History (
history_id INT AUTO_INCREMENT PRIMARY KEY,
watch_date DATE,
user_id INT,
content_id INT,
FOREIGN KEY (user_id) REFERENCES User(user_id),
FOREIGN KEY (content_id) REFERENCES Content(content_id)
);

-- Sample Data
            INSERT INTO Subscription_Plan VALUES (1,'Basic',149),(2,'Standard',499),(3,'Premium',649);

INSERT INTO User (name, email, phone) VALUES
('Rahul Patil', 'rahul@gmail.com', '9876543210'),
('Sneha Kulkarni','sneha@gmail.com', '9823456780'),
('Amit Sharma', 'amit@yahoo.com', '9512345678');

INSERT INTO Content (title, genre, duration) VALUES
('Pushpa 2','Action',180), ('3 Idiots','Comedy',170), ('RRR','Action',182);

INSERT INTO Subscription (start_date,end_date,status,user_id,plan_id)  VALUES
('2025-01-01','2025-01-31','Active', 1, 3),
('2025-01-10','2025-02-09','Active', 2, 2),
('2024-12-01','2024-12-31','Expired', 3, 1);

INSERT INTO Payment (payment_date,amount,payment_method,payment_status,subscription_id)  VALUES
('2025-01-01',649,'UPI', 'Success',1),
('2025-01-10',499,'Card', 'Success',2),
('2024-12-01',149,'Cash', 'Pending',3);

INSERT INTO Watch_History (watch_date,user_id,content_id) VALUES
('2025-01-05',1,1),('2025-01-06',1,3),
('2025-01-11',2,2),('2025-01-12',2,1);
```

3. Program 1 — Display All Users

Aim: Use a cursor to display all users.

Concepts: Cursor, LOOP, IF (control structure), SQLEXCEPTION handler (exception handling). `DELIMITER $$`

```
CREATE PROCEDURE sp_display_users()
BEGIN
-- Exception Handling
DECLARE EXIT HANDLER FOR SQLEXCEPTION
BEGIN
SELECT 'Error occurred while fetching users.' AS Message;
END;

-- Variable Declaration
DECLARE v_id INT;
DECLARE v_name VARCHAR(100);
DECLARE v_email VARCHAR(150);
DECLARE v_done INT DEFAULT FALSE;

-- Cursor Declaration
DECLARE cur_users CURSOR FOR
SELECT user_id, name, email FROM User;
```

```
-- NOT FOUND Handler (Control Structure support)
DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_done = TRUE;

OPEN cur_users;

user_loop: LOOP
FETCH cur_users INTO v_id, v_name, v_email;

-- Control Structure: IF to exit loop
IF v_done THEN
LEAVE user_loop;
END IF;

SELECT v_id AS UserID, v_name AS Name, v_email AS Email;
END LOOP user_loop;

CLOSE cur_users;
END$$

DELIMITER ;

CALL sp_display_users();
```

Output:

1	Rahul Patil	rahul@gmail.com
2	Sneha Kulkarni	sneha@gmail.com
3	Amit Sharma	amit@yahoo.com

4. Program 2 — Display Subscriptions with Status Check

Aim: Use a cursor to display subscriptions. Use IF-ELSE to show a message based on status. **Concepts:** Cursor, IF-ELSE (control structure), SQLEXCEPTION handler (exception handling).

```
DELIMITER $$

CREATE PROCEDURE sp_display_subscriptions()
BEGIN
-- Exception Handling
DECLARE EXIT HANDLER FOR SQLEXCEPTION
BEGIN
SELECT 'Error occurred while fetching subscriptions.' AS Message; END;

DECLARE v_sub_id INT;
DECLARE v_name VARCHAR(100);
DECLARE v_plan VARCHAR(50);
DECLARE v_status VARCHAR(20);
DECLARE v_remark VARCHAR(30);
DECLARE v_done INT DEFAULT FALSE;

DECLARE cur_subs CURSOR FOR
SELECT s.subscription_id, u.name, sp.plan_name, s.status FROM  Subscription s
JOIN User u ON s.user_id = u.user_id
JOIN Subscription_Plan sp ON s.plan_id = sp.plan_id; DECLARE  CONTINUE HANDLER FOR NOT FOUND SET

v_done = TRUE;

OPEN cur_subs;
```

```
sub_loop: LOOP
FETCH cur_subs INTO v_sub_id, v_name, v_plan, v_status;

IF v_done THEN
LEAVE sub_loop;
END IF;

-- Control Structure: IF-ELSE based on status
IF v_status = 'Active' THEN
SET v_remark = 'Currently Active';
ELSE
SET v_remark = 'Not Active';
END IF;

SELECT v_sub_id AS SubID, v_name AS UserName,
v_plan AS Plan, v_status AS Status,
v_remark AS Remark;
END LOOP sub_loop;

CLOSE cur_subs;
END$$

DELIMITER ;

CALL sp_display_subscriptions();
```

Output:

1	Rahul Patil	Premium	Active	Currently Active
2	Sneha Kulkarni	Standard	Active	Currently Active
3	Amit Sharma	Basic	Expired	Not Active

5. Program 3 — Payment Status Check using CASE

Aim: Use a cursor to read all payments. Use CASE to display a message for each payment status. **Concepts:** Cursor, CASE (control structure), SQLEXCEPTION handler (exception handling).

```
DELIMITER $$

CREATE PROCEDURE sp_check_payments()
BEGIN
-- Exception Handling
DECLARE EXIT HANDLER FOR SQLEXCEPTION
BEGIN
```

```
SELECT 'Error occurred while processing payments.' AS Message; END;

DECLARE v_pay_id INT;
DECLARE v_name VARCHAR(100);
DECLARE v_amount DECIMAL(8,2);
DECLARE v_method VARCHAR(20);
DECLARE v_pstatus VARCHAR(20);
DECLARE v_msg VARCHAR(50);
DECLARE v_done INT DEFAULT FALSE;

DECLARE cur_pay CURSOR FOR
SELECT p.payment_id, u.name, p.amount,
p.payment_method, p.payment_status
FROM Payment p
JOIN Subscription s ON p.subscription_id = s.subscription_id JOIN
User u ON s.user_id = u.user_id;

DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_done = TRUE;

OPEN cur_pay;

pay_loop: LOOP
FETCH cur_pay INTO v_pay_id, v_name, v_amount,
v_method, v_pstatus;

IF v_done THEN
LEAVE pay_loop;
END IF;

-- Control Structure: CASE on payment status
CASE v_pstatus
WHEN 'Success' THEN SET v_msg = 'Payment Received';
WHEN 'Pending' THEN SET v_msg = 'Awaiting Confirmation'; WHEN
'Failed' THEN SET v_msg = 'Payment Failed - Retry'; ELSE SET v_msg =  'Unknown Status';
END CASE;

SELECT v_pay_id AS PaymentID,
v_name AS UserName,
v_amount AS Amount,
v_method AS Method,
v_pstatus AS Status,
v_msg AS Message;
END LOOP pay_loop;
CLOSE cur_pay;
END$$

DELIMITER ;

CALL sp_check_payments();
```

Output:

1	Rahul Patil	649.00	UPI	Success	Payment Received
2	Sneha Kulkarni	499.00	Card	Succes	Payment Received

				s	
3	Amit Sharma	149.00	Cash	Pending	Awaiting Confirmation

6. Program 4 — Watch History with WHILE Loop

Aim: Use a cursor with a WHILE loop to read watch history and count total watches. **Concepts:** Cursor, WHILE loop (control structure), SQLEXCEPTION handler (exception handling).

```
DELIMITER $$

CREATE PROCEDURE sp_watch_history_report()
BEGIN
-- Exception Handling
DECLARE EXIT HANDLER FOR SQLEXCEPTION
BEGIN
SELECT 'Error occurred while reading watch history.' AS Message; END;

DECLARE v_uname VARCHAR(100);
DECLARE v_title VARCHAR(200);
DECLARE v_date DATE;
DECLARE v_count INT DEFAULT 0;
DECLARE v_done INT DEFAULT FALSE;

DECLARE cur_wh CURSOR FOR
SELECT u.name, c.title, wh.watch_date
FROM Watch_History wh
JOIN User u ON wh.user_id = u.user_id
JOIN Content c ON wh.content_id = c.content_id
ORDER BY wh.watch_date DESC;

DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_done = TRUE;

OPEN cur_wh;

-- Control Structure: WHILE loop
WHILE v_done = FALSE DO
FETCH cur_wh INTO v_uname, v_title, v_date;
IF v_done = FALSE THEN
SET v_count = v_count + 1;
SELECT v_count AS SrNo,
v_uname AS UserName,
v_title AS ContentTitle,
v_date AS WatchDate;
END IF;
END WHILE;

CLOSE cur_wh;

-- Show total count
SELECT v_count AS TotalWatchRecords;
END$$

DELIMITER ;

CALL sp_watch_history_report();
```

Output:

1	Sneha Kulkarni	Pushpa 2	2025-01-12
2	Sneha Kulkarni	3 Idiots	2025-01-11
3	Rahul Patil	RRR	2025-01-06
4	Rahul Patil	Pushpa 2	2025-01-05
		TotalWatchRecords	4

7. Program 5 — Validate and Insert Subscription (SIGNAL)

Aim: Use a cursor to check if a user already has an active subscription before inserting a new one. Raise an error using SIGNAL if duplicate found.

Concepts: Cursor, IF (control structure), SIGNAL SQLSTATE + DECLARE HANDLER (exception handling).

```
DELIMITER $$

CREATE PROCEDURE sp_add_subscription(
IN p_user_id INT,
IN p_plan_id INT,
OUT p_result VARCHAR(100)
)
BEGIN
-- Exception Handling: catch any DB error
DECLARE EXIT HANDLER FOR SQLEXCEPTION
BEGIN
SET p_result = 'ERROR: Could not create subscription.';
ROLLBACK;
END;
DECLARE v_existing INT DEFAULT 0;
DECLARE v_sub_id INT;
DECLARE v_status VARCHAR(20);
DECLARE v_done INT DEFAULT FALSE;

-- Cursor: check existing subscriptions for this user
DECLARE cur_check CURSOR FOR
SELECT subscription_id, status
FROM Subscription
WHERE user_id = p_user_id;

DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_done = TRUE;

OPEN cur_check;

check_loop: LOOP
FETCH cur_check INTO v_sub_id, v_status;

IF v_done THEN
LEAVE check_loop;
END IF;

-- Control Structure: IF to check active subscription
IF v_status = 'Active' THEN
SET v_existing = 1;
```

```
LEAVE check_loop;
END IF;
END LOOP check_loop;

CLOSE cur_check;

-- Exception Handling: SIGNAL if already subscribed
IF v_existing = 1 THEN
SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT = 'User already has an active subscription.'; ELSE  START TRANSACTION;
INSERT INTO Subscription (start_date, end_date, status, user_id, plan_id) VALUES  (CURRENT_DATE,
DATE_ADD(CURRENT_DATE, INTERVAL 30 DAY), 'Active', p_user_id, p_plan_id);
COMMIT;
SET p_result = CONCAT('SUCCESS: Subscription ID = ', LAST_INSERT_ID()); END  IF;
END$$

DELIMITER ;

-- Test: user 3 has Expired subscription, so new one can be added
CALL sp_add_subscription(3, 2, @res); SELECT @res;

-- Test: user 1 already has Active subscription -> error raised
CALL sp_add_subscription(1, 2, @res); SELECT @res;
```

Output:

CALL sp_add_subscription(3, 2, @res)	SUCCESS: Subscription ID = 4
CALL sp_add_subscription(1, 2, @res)	ERROR: User already has an active subscription.